

Sequence-to-Sequence (Seq2Seq) Models

Previous session

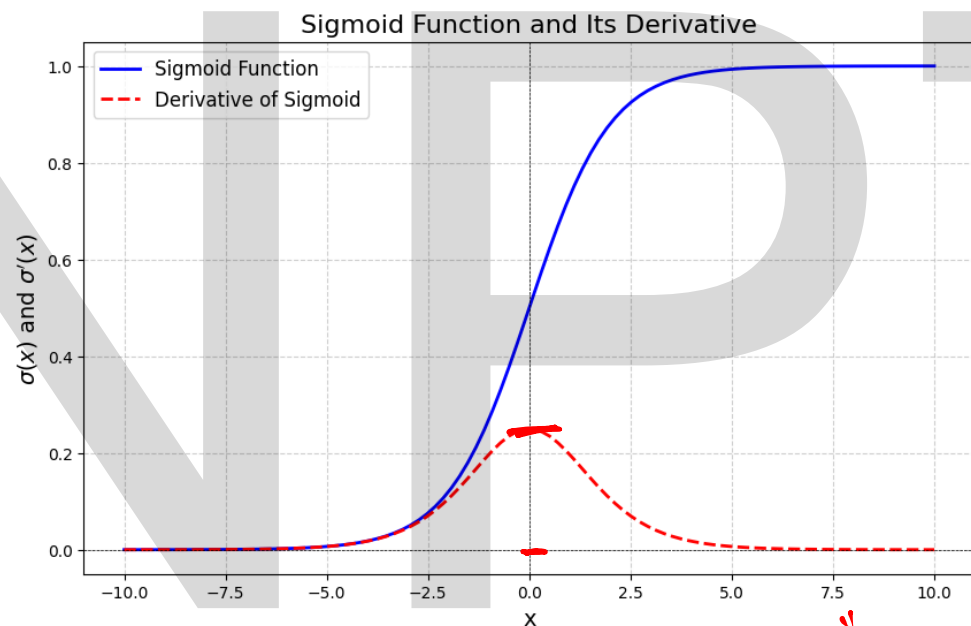
- Numerical Example: Forward Computation in RNN (step-by-step)

NLP TUTOR

Today's Session Overview

- Derivative of sigmoid
- Vanishing gradient problem in deep neural network
- Exploding gradient problem in deep neural network

Derivative of Sigmoid



"0 to 0.25"

$$\sigma(x) = f(x) = \frac{1}{1 + e^{-x}}$$

$$\begin{aligned} \frac{\partial}{\partial x} \left(\frac{1}{1 + e^{-x}} \right) &= -\frac{1}{(1 + e^{-x})^2} \cdot \frac{\partial}{\partial x} (1 + e^{-x}) \cdot (-1) \\ &= \frac{e^{-x}}{(1 + e^{-x})^2} \end{aligned}$$

Split:

$$\begin{aligned} &= \frac{1}{(1 + e^{-x})} \cdot \frac{e^{-x}}{(1 + e^{-x})} \\ &= \sigma(x) \cdot (1 - \sigma(x)) \end{aligned}$$

Now prove: $(1 - \sigma(x))$ is $\frac{e^{-x}}{(1 + e^{-x})}$

$$(1 - \sigma(x)) = 1 - \frac{1}{(1 + e^{-x})}$$

$$\frac{1 + e^{-x} - 1}{1 + e^{-x}} = \frac{e^{-x}}{1 + e^{-x}}$$

Therefore derivative of sigmoid is $\sigma(x) \cdot (1 - \sigma(x))$

For any given dataset, when we build a deep learning model—whether it is an ANN, CNN, or RNN—the most critical phase is **training**.

During training, the model learns **optimized parameters**:

- **ANN**: weights and biases ✓
- **CNN**: convolution kernels (filters), weights, and biases ✓
- **RNN**: input, recurrent, and output weights along with biases

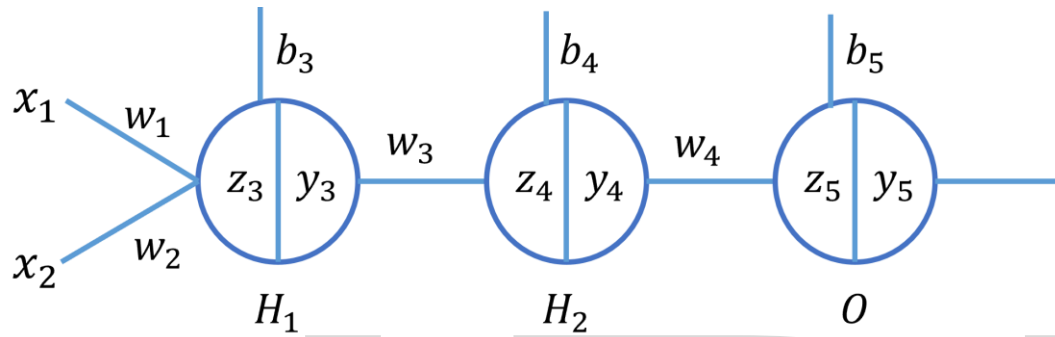
The success of a deep neural network depends on how effectively these parameters are updated during training using **backpropagation and gradient descent**.

If the **weight updates occur very slowly**, or if they become **unstable**, the model fails to learn meaningful representations—making the depth of the network ineffective. }

Importantly, this issue **does not arise from the input data**, since the input is fixed and given. Instead, the problem originates from **how the model is designed**, particularly:

- the **depth of the network**, and ✓
- the **activation functions used in the hidden layers**. ✓

Thus, the **choice of activation function in hidden layers plays a central role** in determining whether learning is stable, slow, or unstable.



$$x_1 = 0.45, x_2 = 0.5$$

$$w_1 = 0.2, w_2 = 0.4, w_3 = 0.2, w_4 = 0.7$$

$$b_3 = 0.2, b_4 = 0.4, b_5 = 0.7$$

$$y = 1$$

Activation function used is Sigmoid @ hidden and @ output layer

At hidden layer 1:

$$z_3 = x_1 w_1 + x_2 w_2 + b_3 = 0.49$$

$$y_3 = \sigma(z_3) = 0.6201$$

At hidden layer 2:

$$z_4 = y_3 w_3 + b_4 = 0.5240$$

$$y_4 = \sigma(z_4) = 0.6281$$

At output layer:

$$z_5 = y_4 w_4 + b_5 = 1.1397$$

$$y_5 = \sigma(z_5) = \underline{0.7577}$$

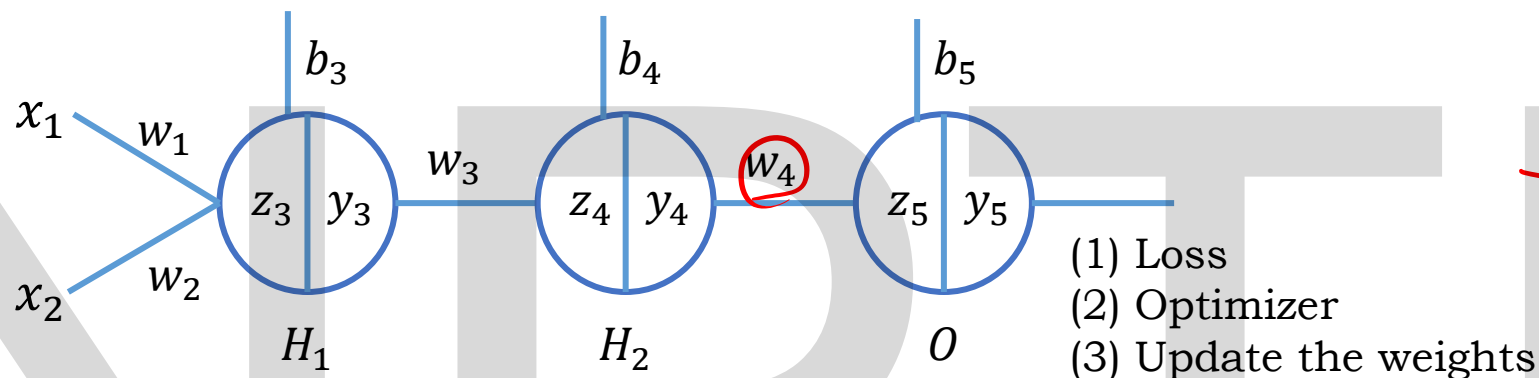
$$BCE = -\frac{1}{m} \sum_{i=1}^m y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$$

$$\frac{\partial L}{\partial \hat{y}_i} = -\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i}\right) = \underline{-1.3193}$$

Here $\hat{y}_i = y_5$

$$\begin{cases} y_i = 1 \\ \hat{y}_i = 0.7577 \end{cases}$$

Vanishing gradient in Deep Learning



$$w_4 = 0.7, y_4 = 0.6281, y_5 = 0.7577$$

der of Loss = -1.3193

$$w_4 = w_4 - \eta \frac{\partial L}{\partial w_4}$$

$$= 0.7 - 0.01(-0.1520)$$

$$= \underline{0.7015}$$

$w_{4(old)}$ is almost equal to $w_{4(new)}$

$$\frac{\partial L}{\partial w_4} = \frac{\partial L}{\partial y_5} \cdot \frac{\partial y_5}{\partial z_5} \cdot \frac{\partial z_5}{\partial w_4}$$

$$= -\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i}\right) \cdot \underbrace{y_5(1 - y_5)}_{0 \text{ to } 0.25} \cdot y_4$$

$$= -1.3193 * \underline{0.1835} * 0.6281$$

$$= \underline{-0.1520}$$

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

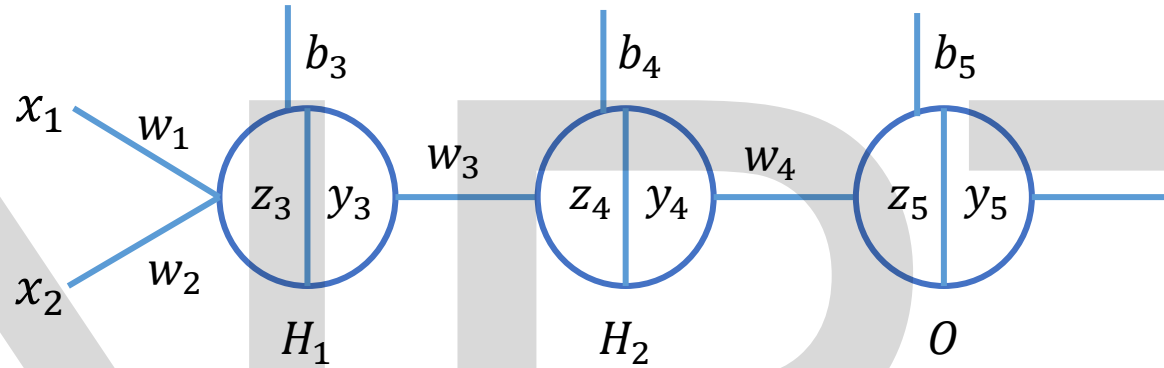
$$\frac{\partial L}{\partial b_5} = \frac{\partial L}{\partial y_5} \cdot \frac{\partial y_5}{\partial z_5} \cdot \frac{\partial z_5}{\partial b_5}$$

$$z_5 = y_4 w_4 + b_5$$

$$y_5 = \sigma(z_5)$$

$$\frac{\partial y_5}{\partial z_5} = \sigma(z_5)(1 - \sigma(z_5))$$

Vanishing gradient in Deep Learning



$$w_4 = 0.7, y_4 = 0.6281, y_5 = 0.7577, \\ y_3 = 0.6201 \\ \text{der of Loss} = -1.3193$$

$$w_3 = w_3 - \eta \frac{\partial L}{\partial w_3} \\ = 0.2 - 0.01(-0.0245) \\ = 0.2002$$

$w_{3(old)}$ is almost equal to $w_{3(new)}$

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial y_5} \cdot \frac{\partial y_5}{\partial z_5} \cdot \frac{\partial z_5}{\partial y_4} \cdot \frac{\partial y_4}{\partial z_4} \cdot \frac{\partial z_4}{\partial w_3} \\ = - \left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i} \right) \cdot \underbrace{y_5(1 - y_5)}_{0 \text{ to } 0.25} \cdot w_4 \cdot \underbrace{y_4(1 - y_4)}_{0 \text{ to } 0.25} \cdot y_3 \\ = -1.3193 * 0.1835 * 0.7 * 0.2336 * 0.6201 \\ = -0.0245$$

$$W_{3(new)} = W_{3(old)} - \eta \frac{\partial L}{\partial W_3}$$

$$z_5 = y_4 w_4 + b_5 \\ y_5 = \sigma(z_5)$$

$$\frac{\partial y_5}{\partial z_5} = \sigma(z_5)(1 - \sigma(z_5))$$

$$z_4 = y_3 w_3 + b_4 \\ y_4 = \sigma(z_4)$$

$$\frac{\partial y_4}{\partial z_4} = \sigma(z_4)(1 - \sigma(z_4))$$

Vanishing gradient in Deep Learning

Diagram of a 3-layer neural network with input layer H_1 , hidden layer H_2 , and output layer O .

Forward pass values:

$$w_3 = 0.2, w_4 = 0.7, y_4 = 0.6281, y_5 = 0.7577, y_3 = 0.6201, x_1 = 0.45$$

der of Loss = -1.3193

Backward pass (gradient flow):

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_5} \cdot \frac{\partial y_5}{\partial z_5} \cdot \frac{\partial z_5}{\partial y_4} \cdot \frac{\partial y_4}{\partial z_4} \cdot \frac{\partial z_4}{\partial y_3} \cdot \frac{\partial y_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_1}$$

Gradients at each layer:

- Output layer O : $\frac{\partial y_5}{\partial z_5} = \sigma(z_5)(1 - \sigma(z_5))$
- Hidden layer H_2 : $\frac{\partial y_4}{\partial z_4} = \sigma(z_4)(1 - \sigma(z_4))$
- Input layer H_1 : $\frac{\partial y_3}{\partial z_3} = \sigma(z_3)(1 - \sigma(z_3))$

Forward pass equations:

$$z_5 = y_4 w_4 + b_5$$

$$y_5 = \sigma(z_5)$$

$$z_4 = y_3 w_3 + b_4$$

$$y_4 = \sigma(z_4)$$

$$z_3 = x_1 w_1 + x_2 w_2 + b_3$$

$$y_3 = \sigma(z_3)$$

Backward pass calculation for w_1 :

$$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1}$$

$$= 0.2 - 0.01(-0.0008)$$

$$= 0.200008$$

w_1 is almost equal to w_1 (new)

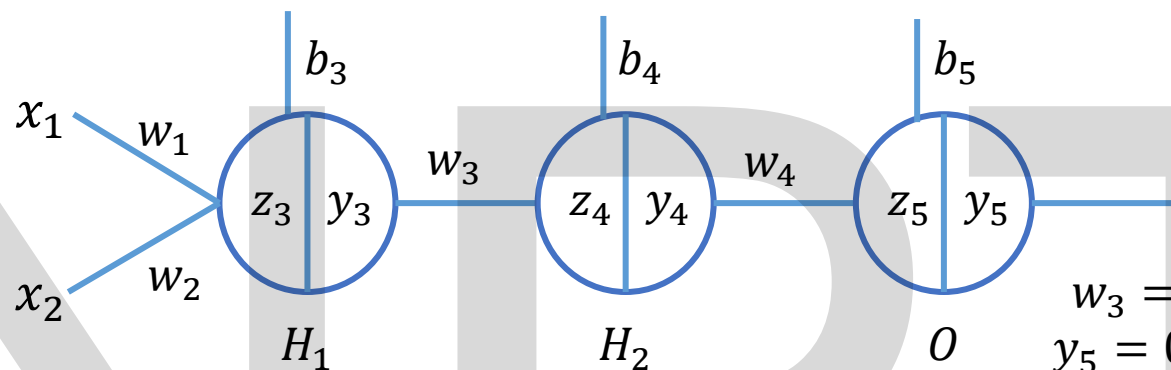
Final calculation for w_1 :

$$= -1.3193 * 0.1835 * 0.7 * 0.2336 * 0.2 * 0.2356 * 0.45$$

$$= -0.0008$$

Vanishing gradient in Deep Learning

$$W_{2(new)} = W_{2(old)} - \eta \frac{\partial L}{\partial W_2}$$



$$w_3 = 0.2, w_4 = 0.7, y_4 = 0.6281, y_5 = 0.7577, y_3 = 0.6201, x_2 = 0.5$$

der of Loss = -1.3193

$$z_5 = y_4 w_4 + b_5$$

$$y_5 = \sigma(z_5)$$

$$\frac{\partial y_5}{\partial z_5} = \sigma(z_5)(1 - \sigma(z_5))$$

$$z_4 = y_3 w_3 + b_4$$

$$y_4 = \sigma(z_4)$$

$$\frac{\partial y_4}{\partial z_4} = \sigma(z_4)(1 - \sigma(z_4))$$

$$z_3 = x_1 w_1 + x_2 w_2 + b_3$$

$$y_3 = \sigma(z_3)$$

$$\frac{\partial y_3}{\partial z_3} = \sigma(z_3)(1 - \sigma(z_3))$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial y_5} \cdot \frac{\partial y_5}{\partial z_5} \cdot \frac{\partial z_5}{\partial y_4} \cdot \frac{\partial y_4}{\partial z_4} \cdot \frac{\partial z_4}{\partial y_3} \cdot \frac{\partial y_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_2}$$

$$w_2 = w_2 - \eta \frac{\partial L}{\partial w_2} = - \left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i} \right) \cdot y_5(1 - y_5) \cdot w_4 \cdot y_4(1 - y_4) \cdot w_3 \cdot y_3(1 - y_3) \cdot x_2$$

$$= 0.4 - 0.01(-0.0009)$$

$$= 0.400009$$

0 to 0.25

0 to 0.25

0 to 0.25

$$= -1.3193 * 0.1835 * 0.7 * 0.2336 * 0.2 * 0.2356 * 0.5$$

$$= -0.0009$$

$w_{2(old)}$ is almost equal to $w_{2(new)}$

In general

For a deep network with L layers:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_5} \cdot \frac{\partial y_5}{\partial z_5} \cdot \frac{\partial z_5}{\partial y_4} \cdot \frac{\partial y_4}{\partial z_4} \cdot \frac{\partial z_4}{\partial y_3} \cdot \frac{\partial y_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_1}$$

0 to 0.25

This shows that the gradient at early layers is a **product of many small terms**.

Key Property of Sigmoid:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), 0 < \sigma'(z) \leq 0.25$$

$$\frac{\partial y}{\partial z} = \sigma'(z) \leq 0.25$$

0 to 0.25

In deep neural networks using **sigmoid activation or tanh**, gradients reaching the early layers become extremely small due to repeated multiplication of activation derivatives (each ≤ 0.25 for sigmoid and ≤ 1 for tanh) across multiple layers.

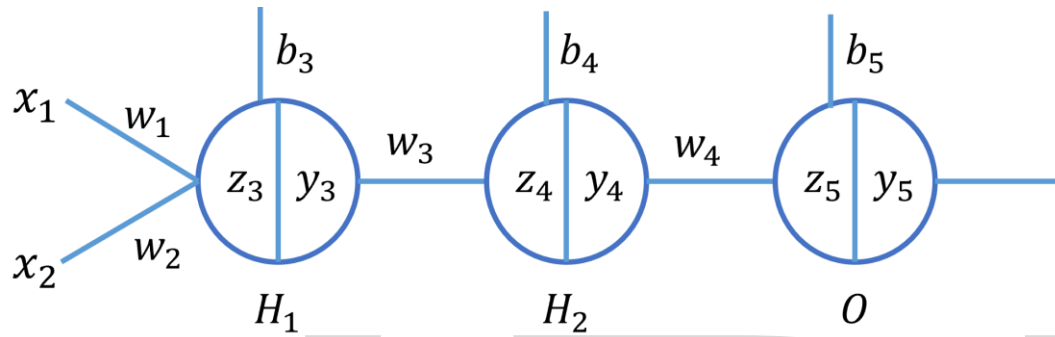
As a result, **weights in the initial layers update very slowly**, causing:

- Slow convergence
- Poor learning of early features
- Training failure in very deep networks

This effect becomes more severe as the depth (L) of the network increases.

This phenomenon is called the **Vanishing Gradient Problem**.

This problem is **significantly reduced by using ReLU activation instead of sigmoid**.



Activation function used is ReLU @ hidden and Sigmoid @ output layer

At hidden layer 1:

$$z_3 = x_1 w_1 + x_2 w_2 + b_3 = 0.49$$

$$y_3 = \text{ReLU}(z_3) = 0.49$$

At hidden layer 2:

$$z_4 = y_3 w_3 + b_4 = 0.498$$

$$y_4 = \text{ReLU}(z_4) = 0.498$$

At output layer:

$$z_5 = y_4 w_4 + b_5 = 1.0486$$

$$\underline{y_5} = \sigma(z_5) = \underline{0.7405}$$

$$x_1 = 0.45, x_2 = 0.5$$

$$w_1 = 0.2, w_2 = 0.4, w_3 = 0.2, w_4 = 0.7$$

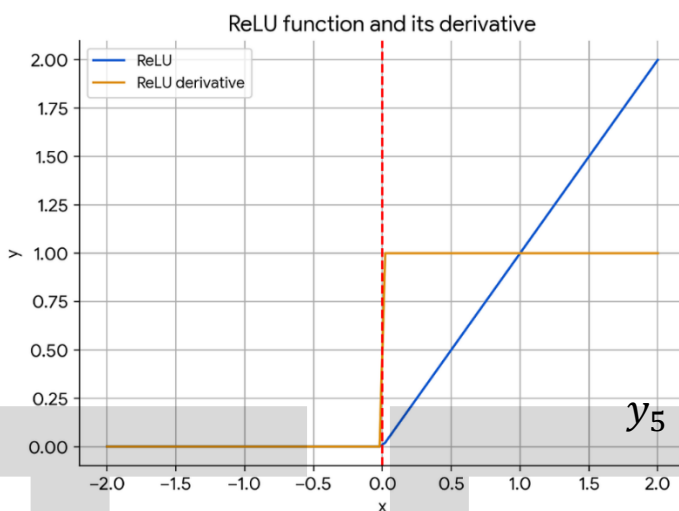
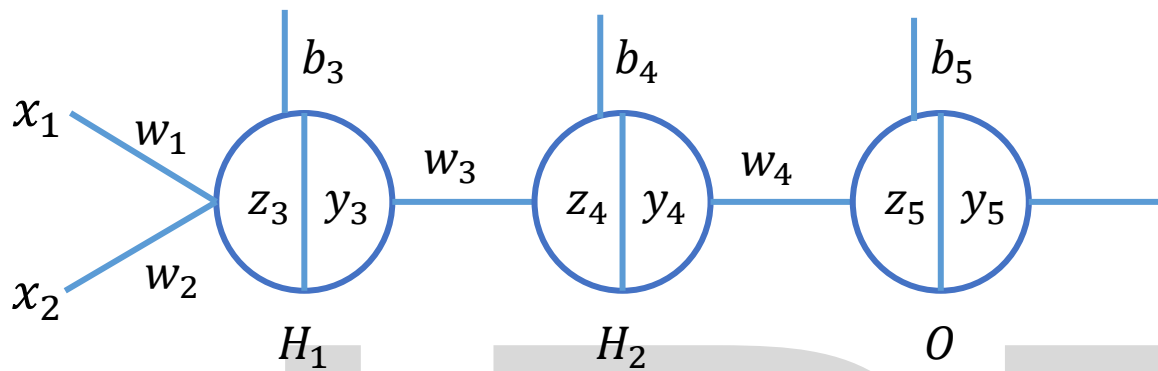
$$b_3 = 0.2, b_4 = 0.4, b_5 = 0.7$$

$$y = \underline{1}$$

$$BCE = -\frac{1}{m} \sum_{i=1}^m y_i \log(\hat{y}_i) + (1 - y) \log(1 - \hat{y}_i)$$

$$\frac{\partial L}{\partial \hat{y}_i} = -\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i}\right) = -\underline{1.3504}$$

Here $\hat{y}_i = y_5$



If ReLU is used then the derivative of ReLU values are 0 or 1.

$$\begin{cases} 1 & \text{if } Z > 0 \\ 0 & \text{if } Z \leq 0 \end{cases}$$

$$y_5 = 0.7405, y_4 = 0.498, y_3 = 0.49$$

$$w_4 = 0.7$$

$$\begin{aligned} \frac{\partial L}{\partial w_4} &= \frac{\partial L}{\partial y_5} \cdot \frac{\partial y_5}{\partial z_5} \cdot \frac{\partial z_5}{\partial w_4} \\ &= -\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i}\right) \cdot y_5(1 - y_5) \cdot y_4 \\ &= -1.3504 * 0.1921 * 0.498 \\ &= -0.12918 \end{aligned}$$

$$\begin{aligned} z_5 &= y_4 w_4 + b_5 \\ y_5 &= \sigma(z_5) \end{aligned}$$

$$\frac{\partial y_5}{\partial z_5} = y_5(1 - y_5)$$

$$\begin{aligned} z_4 &= y_3 w_3 + b_4 \\ y_4 &= \max(0, z_4) \end{aligned}$$

$$\begin{aligned} w_4 &= w_4 - \eta \frac{\partial L}{\partial w_4} \\ &= 0.7 - 0.01(-0.12918) \\ &= 0.70129 \end{aligned}$$

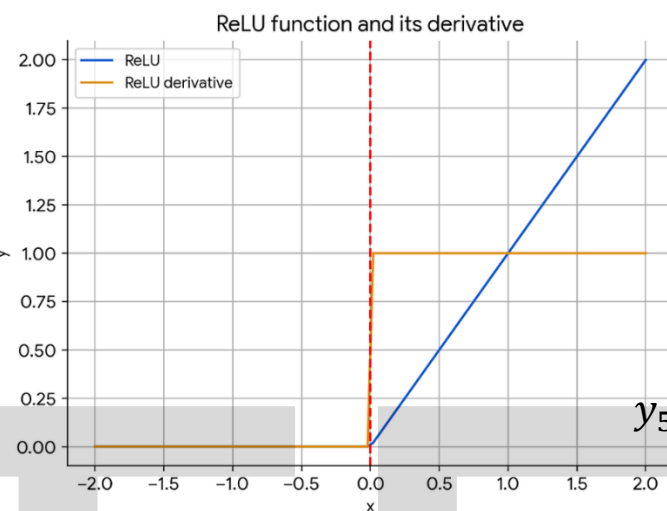
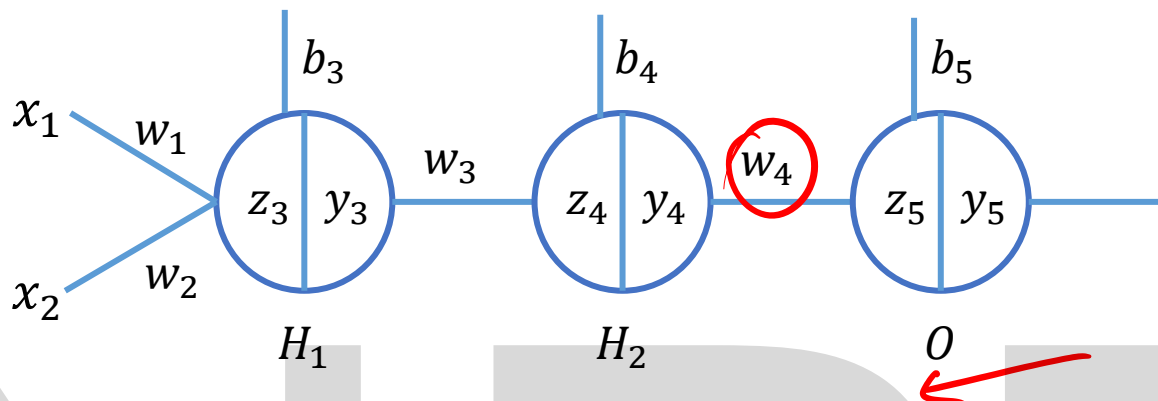
$$\begin{aligned} \frac{\partial L}{\partial w_3} &= \frac{\partial L}{\partial y_5} \cdot \frac{\partial y_5}{\partial z_5} \cdot \frac{\partial z_5}{\partial y_4} \cdot \frac{\partial y_4}{\partial z_4} \cdot \frac{\partial z_4}{\partial w_3} \\ &= -\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i}\right) \cdot y_5(1 - y_5) \cdot w_4 \cdot \mathbf{1} \cdot y_3 \\ &= -1.3504 * 0.1921 * 0.7 * \mathbf{1} * 0.49 \\ &= -0.08897 \end{aligned}$$

$$\begin{aligned} w_3 &= w_3 - \eta \frac{\partial L}{\partial w_3} \\ &= 0.2 - 0.01(-0.08897) \\ &= 0.200889 \end{aligned}$$

If the **ReLU derivative becomes 0**, then the **entire gradient becomes 0**, and therefore:

$$w_{new} = w_{old}$$

There will be **no weight update at all**.



If ReLU is used then the derivative of ReLU values are 0 or 1.

$$\begin{cases} 1 & \text{if } Z > 0 \\ 0 & \text{if } Z \leq 0 \end{cases}$$

$$y_5 = 0.7405, w_3 = 0.2, x_1 = 0.45, w_4 = 0.7$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_5} \cdot \frac{\partial y_5}{\partial z_5} \cdot \frac{\partial z_5}{\partial y_4} \cdot \frac{\partial y_4}{\partial z_4} \cdot \frac{\partial z_4}{\partial y_3} \cdot \frac{\partial y_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_1}$$

$$= -\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i}\right) \cdot y_5(1 - y_5) \cdot w_4 \cdot 1 \cdot w_3 \cdot 1 \cdot x_1$$

$$= -1.3504 * 0.1921 * 0.7 * \underline{1} * 0.2 * \underline{1} * 0.45$$

Handwritten: 0.01 (above 1), 0.01 (above 1)

$$= -0.01634$$

$$w_1 = 0.2 - 0.01(-0.01634)$$

$$= \underline{0.200163}$$

If the **ReLU derivative becomes 0**, then the **entire gradient becomes 0**, and therefore:

$$w_{new} = w_{old}$$

There will be **no weight update at all**.

To **avoid zero gradients in the negative region**. Leaky ReLU is defined as:

$$\text{Leaky ReLU}(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases} \text{ where } \alpha \approx \underline{0.01}$$

Derivative:

$$\text{Leaky ReLU}'(z) = \begin{cases} 1 & z > 0 \\ \alpha & z \leq 0 \end{cases}$$

old	Sigmoid	ReLU
<u>w₁ = 0.2</u>	<u>0.200008</u>	<u>0.200162</u>
<u>w₂ = 0.4</u>	<u>0.400009</u>	<u>0.400181</u>
<u>w₃ = 0.2</u>	<u>0.2002</u>	<u>0.20088</u>
<u>w₄ = 0.7</u>	<u>0.7015</u>	<u>0.7012</u>

Exploding gradient problem

$$\begin{aligned}\frac{\partial L}{\partial w_1} &= \frac{\partial L}{\partial y_5} \cdot \frac{\partial y_5}{\partial z_5} \cdot \frac{\partial z_5}{\partial y_4} \cdot \frac{\partial y_4}{\partial z_4} \cdot \frac{\partial z_4}{\partial y_3} \cdot \frac{\partial y_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_1} \\ &= -(y - y_5) \cdot \underbrace{y_5(1 - y_5)} \cdot \underbrace{w_4} \cdot \underbrace{y_4(1 - y_4)} \cdot \underbrace{w_3} \cdot \underbrace{y_3(1 - y_3)} \cdot x_1\end{aligned}$$

For sigmoid activation:

$$0 < y(1 - y) \leq 0.25$$

If **weights are large**, for example:

$$w_3 = 50, w_4 = 50$$

Then the gradient becomes:

$$\text{Very Large} \Rightarrow \frac{\partial L}{\partial w_1} \uparrow \uparrow$$

Even though sigmoid derivatives are ≤ 0.25 , if the weights are very large, the product of large weights dominates and causes exploding gradients.

Weight Update Rule:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

If gradient is very large $\Rightarrow$ Weight update $\checkmark$
becomes very large $\Rightarrow$ **Loss starts oscillating**
or diverging $\Rightarrow$ Training becomes unstable
 $\Rightarrow$ Model fails to converge

Next Session

- Backpropagation Through Time (BTT) ✓
- Challenges such as Vanishing and Exploding Gradients in RNN
- How to fix exploding gradients

NIPTELL

Thank you